NTNU

elsys
ELEKTRONISK SYSTEMDESIGN & INNOVASJON

Project Report EDK

Title: Implementation And Theory Of Classifications Algorithms

Authors: Ulrik Garmark Anker, Åmund Villumstad Velle

Version: 1.0 | Date: April 30, 2026

1 Summary

In this report we were tasked with implementing a linear classifier on the Iris dataset split in two different ways, before reducing features to see if it improves performance. we were also tasked with testing an NN-, a Kmeans template based NN-, and a KNN-classifer on the MNIST dataset and compare them. For the Iris task we achieved error rates of 0 and 0.0333 and noted a slight deterioration when removing features. For the MNIST dataset the KNN-classifier achieved an error rate of 0.0306, the NN achieved an error rate of 0.0309, and the Kmeans based achieved an error rate of 0.1516.

Contents

1 Summary **1**

2 Introduction **2**

3 Theory **2**
3.1 Evaluating Classifiers . . . 2
3.2 Linear Classifiers . . . 2
3.3 Template Based Classifiers . . . 4

4 Task **5**
4.1 Iris Dataset . . . 5
4.2 MNIST Dataset . . . 5

5 Implementation **5**
5.1 Iris Implementation . . . 5
5.2 Iris Results . . . 7
5.3 MNIST Implementation . . . 9
5.4 MNIST Results . . . 10

6 Conclusion **14**

References **15**

2 Introduction

In this report we will describe the theory and implementation of two families of classification algorithms, one for the Iris and one for the MNIST dataset. The use of classification algorithms to classify data is a very important issue within data processing and machine learning with applications in many fields ranging from medicine to radio. In chapter 3 we will describe the theory behind each classifier, in chapter 4 we will describe the task, in chapter 5 we will describe our implementation, and finally in chapter 6 we give our conclusion.

3 Theory

In this section we will go over the basic theory behind the classifiers we have used. First we will discuss the parameters we use to evaluate classifiers, we will then explain the linear classifier and then the template based classifier.

3.1 Evaluating Classifiers

In evaluating the performance of our classifiers we have some important metrics. The simplest one is the error rate, it is simply the percentage of classifications that are wrong. This gives us information about the overall performance however it does not indicate what errors are most common or why it commits those errors.

To solve this we introduce the confusion matrix. It consists of a table in which the rows represent the true value and the columns represents the guessed values. An example of this is shown under in figure 1. This presentation of data allows us to quickly see which labels the classifier is struggling with and what it is confusing it for.

Guessed / True	label 1	label 2	label 3
label 1	20	0	0
label 2	0	20	3
label 3	3	0	20

Figure 1: Confusion matrix example

3.2 Linear Classifiers

Assuming our problem is linearly separable a linear classifier is a good option. A discriminant classifier is defined by a discriminant function $g_i(x)$ and the decision rule is given in equation 1.

$$x \in \omega_j \iff g_j(x) = \max_i g_i(i) \text{ [1]} \tag{1}$$

The linear discriminant function for $C > 2$ classes and D features is given by:

$$g(x) = Wx + w_0 \text{ [1]} \tag{2}$$

Where W is a $C \times D$ matrix and g and w_0 are vectors of dimension C. We can redefine W to be $[W, w_0]$ and x to be $[x^T, 1]^T$ and thus rewrite the discriminant function as:

$$g(x) = Wx \tag{3}$$

To create a useful classifier we must find values for W that usefully distinguish our classes. We achieve this by calculating the MSE then doing gradient descent iteratively. We define the Mean Square Error to be:

$$MSE = \frac{1}{2}\sum_{k=1}^{N}(g_k - t_k)^T(g_k - t_k) \text{ [1]} \tag{4}$$

where t_k is the target vector for the feature x_k i.e a vector with all 0 except for the index that corresponds to the true label which is 1. As g_k is continuous and we desire the output to be binary we redefine g_k to be the sigmoid function of $z_k(x_k) = Wx_k$. We then have the equation:

$$g_{ik} = \text{Sigmoid}(x_{ik}) = \frac{1}{1 - e^{-z_{ik}}}, \quad i = 1, \ldots, C \text{ [1]} \tag{5}$$

The use of a sigmoid function squashes the output to be closer to a Heavyside function while still having a derivative. We must then take the gradient of the MSE to in order to approach a local minima. Applying the chain rule we have:

$$\nabla_W MSE = \sum_{k=1}^{N} \nabla_{g_k} MSE \nabla_{z_k} g_k \nabla_W z_k \text{ [1]} \tag{6}$$

where

$$\nabla_{g_k} MSE = g_k - t_k \tag{7}$$

$$\nabla_{z_k} g_k = g_k \circ (1 - g_k) \tag{8}$$

$$\nabla_W z_k = x_k^T \tag{9}$$

The $\circ$ operator is the elementwise multiplication operator. We finally then have the equation:

$$\nabla_W MSE = \sum_{k=1}^{N}[(g_k - t_k) \circ g_k \circ (1 - g_k)]x_k^T \tag{10}$$

By moving opposite of this we approach a local minima so we can write our mth iteration of W as:

$$W(m) = W(m-1) - \alpha \nabla_W MSE \text{ [1]} \tag{11}$$

The parameter α must be chosen carefully as if it is too large the training may not converge and if it is too small the training will take too long. A common option for optimizing the parameter α is a grid search in which an entire range is tested and the best performing value is then chosen. Ideally one would do grid search on a separate evaluation test set as to prevent data leakage.

3.3 Template Based Classifiers

3.3.1 Nearest Neighbour Classifier

A Nearest Neighbour Classifier, henceforth referred to as an NN-Classifier, is a non-parametric supervised learning method. The basic idea behind the classifier is to compare an input x to a template set. This template set can be created from a set of training data and was for this project initially created by including the entire training set such that each element of the training set is included as a reference. The class of x is chosen as the reference closest to x. There are different ways to define the closest reference, but for this project we employed euclidean distance as defined by

$$d(x, ref_i) = (x - ref_i)^\top (x - ref_i) \text{ [1]} \tag{12}$$

The NN-classifier calculates the distance above for x for each $ref_i \in$ Template. The class of x is chosen as

$$\text{class}(\arg\min_{ref_i}(d(x, ref_i))) \text{ [1]} \tag{13}$$

3.3.2 K-means based classifier

We also implemented a K-means based NN-classifier. The decision rule for the class of x is the same as for the regular NN-classifier above. The difference lays within the template choice. Instead of using the entire training set as a template, we first clustered the training set into K clusters using K-means clustering. K-means clustering aims to partition the data into k sets $\boldsymbol{S} = \{S_1, S_2, ..., S_k\}$. Formally, the objective is to find the $\boldsymbol{S}$ which minimizes

$$\begin{aligned} \sum_{i}^{k} \sum_{x \in S_i} \|x - \mu_i\|_2^2 \\ \mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x \end{aligned} \tag{14}$$

Essentially, the goal is to find the sets that make up $\boldsymbol{S}$ that minimize the sum of distance between the mean of each set S_i in $\boldsymbol{S}$ and the values of x within S_i [2]. Before performing K-means clustering, the values of x should be normalized such that $x \in \{0, 1\}$.

To use K-means in an NN-classifier, you must then assign a label to each $S_i \in \boldsymbol{S}$. We chose to do this by picking the most common label within each set. From this point, the classifier is operated in the exact same way as the regular NN-classifier.

3.3.3 K Nearest Neighbour Classifier

A K Nearest Neighbour Classifier, henceforth referred to as a KNN-classifier, is an expansion of the regular NN-classifier. Instead of assigning x the same label as its one closest neighbour, x is assigned the label of the most common label amongst its k nearest neighbours. This also means that a KNN-classifier with $k = 1$ is just an NN-classifier [1].

4 Task

4.1 Iris Dataset

The iris flower has three variants in our dataset. Each variant has 50 examples totaling 150 entries in the dataset. Each sample has four unique features, sepal length, sepal width, petal length, and petal width. Our task is to train a linear classifier on the first 30 samples of each type then test on the remaining 20 of each type, to achieve this we must tune our step size α. We will then find the confusion matrix and the error rate of our test. We will also use the last 30 samples of each type for training and first 20 for testing and comment on any differences we observe. In the second part we will produce histograms of our features to see what features have most overlap, remove the feature with most overlap and train our classifier on this new reduced dataset. We will then repeat this, retaining just 2 and 1 features respectively, and compare the resulting error rates and confusion matrices.

4.2 MNIST Dataset

The MNIST dataset consists of 7 000 images of hand drawn digits. We split this into 6 000 digits for the training set, and 1 000 for the test set. Each digit contains a label between zero and nine and consists of a grid of 28×28 pixels of values in the range $\{0, 255\}$. Our task is to first classify the digits using an NN-classifier as described in subsection 3.3.1. Following this we also classify using a Kmeans based NN-classifier as described in subsection 3.3.2 and a KNN-classifier as described in subsection 3.3.3. We find their total error rates, confusion matrices, and plot some of their correctly and incorrectly classified digits. As an addition, we have decided to comment on the classification speed of each classifier.

5 Implementation

5.1 Iris Implementation

For the first task the data was read in from a CSV file into the program before being separated into a test set and a training set. This was done by adding the first 30 samples of each class to the training set and adding the remaining 20 samples to the test set. Another test and training set pair was created by adding the last 30 samples of each class into a training set

and taking the first 20 and adding to a test set. Henceforth these will be referred to as the first and second data set split respectively. The step size parameter α was then tuned by performing a grid search whereby the linear classifier was trained for 1000 iterations on the training set before being tested on the test set. The test accuracy was then plotted against the tested value of α. This produced the graph shown in figure 2.

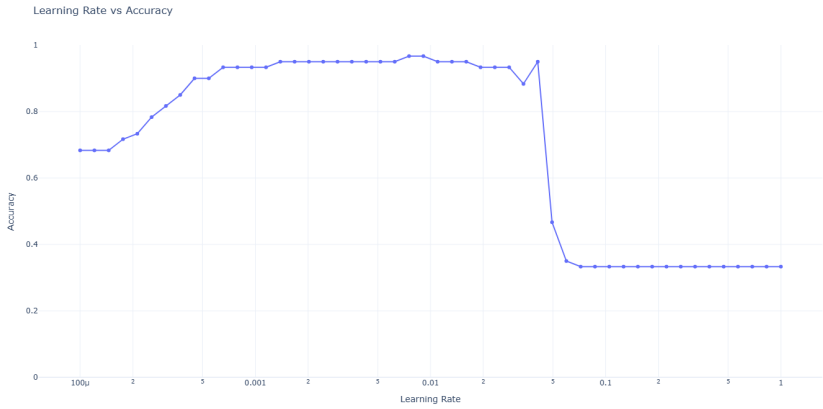


Figure 2: Graph showing attained accuracy against step size

In the graph we can see that the attained accuracy increases before suddenly decreasing to random chance as the large step size causes the training to diverge. From this data it was determined the optimal step size was $\alpha = 0.008$. For evaluating the weights were trained using the optimal step size for 10000 iterations before being tested on the test set. The testing and training was repeated on the second dataset using the same step size.

For the second task a histogram was produced showing the overlap of features. It is shown under in figure 3.

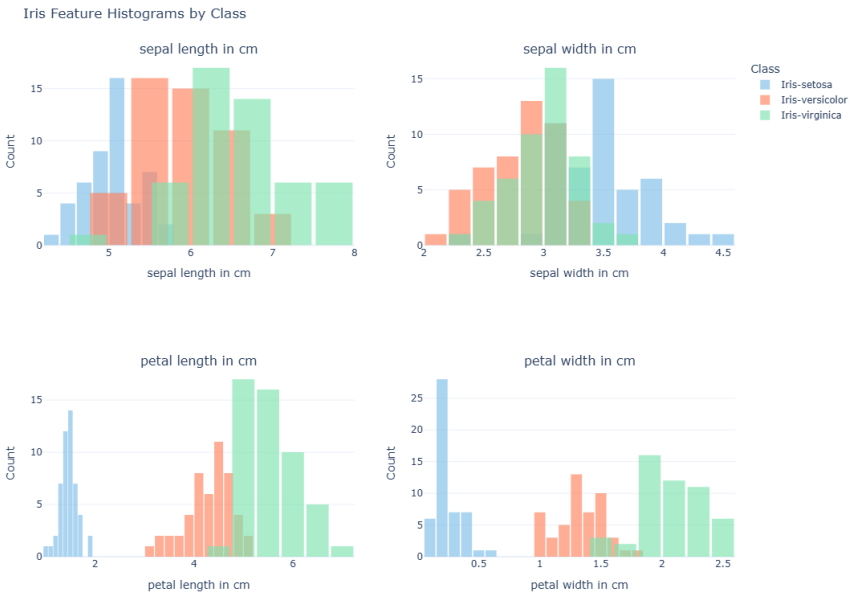


Figure 3: Histogram of iris features

Both datasets were then tested with 10000 training iterations as a benchmark. Features of the datasets were removed in order of most overlap to least overlap before training and testing resulting in a total of 8 runs with features removed in the order sepal width, sepal length and finally petal length. All runs were done using step size 0.008.

5.2 Iris Results

5.2.1 Task 1

The training runs in task 1 attained error rates of 0.0333 for the first split and 0 for the second split. Their corresponding error matrices are shown under in figures 4 and 5

True \ Guessed	Iris-Setosa	Iris-Versicolor	Iris-Virginica
Iris Setosa	20	0	0
Iris-Versicolor	0	18	2
Iris-Virginica	0	0	20

Figure 4: Confusion matrix from the first data set split

True \ Guessed	Iris-Setosa	Iris-Versicolor	Iris-Virginica
Iris Setosa	20	0	0
Iris-Versicolor	0	20	0
Iris-Virginica	0	0	20

Figure 5: Confusion matrix from the second data set split

The low error rates reflect the fact that the different classes are close to linearly separable. The classifier struggles to separate the Iris Versicolor and the Iris Virginica, this makes sense as in figure 3 there is no feature for which there is no overlap. That no error occurred in the second split might indicate that the second test set contain no or at least fewer flowers in the overlap between classes.

5.2.2 Task 2

Table 1 presents the data from the different test runs.

Nr. Features	Data Set Split	Error Rate	Confusion Matrix
4	1	0.0333	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 18 & 2 \\ 0 & 0 & 20 \end{bmatrix}$
4	2	0	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 0 & 20 \end{bmatrix}$
3	1	0.05	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 18 & 2 \\ 0 & 1 & 19 \end{bmatrix}$
3	2	0.0167	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 1 & 19 \end{bmatrix}$
2	1	0.05	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 19 & 1 \\ 0 & 2 & 18 \end{bmatrix}$
2	2	0.0833	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 5 & 15 \end{bmatrix}$
1	1	0.0667	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 18 & 2 \\ 0 & 2 & 18 \end{bmatrix}$
1	2	0.0333	$\begin{bmatrix} 20 & 0 & 0 \\ 0 & 19 & 1 \\ 0 & 1 & 19 \end{bmatrix}$

Table 1: Table of data collected for task 2

For the first data split there was a slight deterioration when the most overlapping feature was removed, no change when the second feature was removed and a final small deterioration for the last feature removed. The second split on the other hand showed a small deterioration for the first feature removed, a marked deterioration for the second one and a slight recovery for the third one. It is clear that the first feature removed some information that the classifier used to distinguish between iris-versicolor and iris-virginica as both runs deteriorated after its removal. The second feature causes some divergence as the second split deteriorates a lot while the first split does not. This indicates that the second split is skewed in some feature causing poor performance on the test set. this is rectified when the third feature is removed indicating that the skewed part of the second split is in the petal length. A larger dataset might rectify these instabilities as depending on how we split our dataset there is some variance which is exacerbated by removing data.

5.3 MNIST Implementation

For each section, the resulting classification was saved as a pickle file. These pickle files were then loaded by a separate program for analysis and visualization of the results. This was done so that the data could be analysed multiple times as the classification time for some of the classifiers was relatively long. The python script used to perform the analysis calculates the total error rate, finds and plots the confusion matrix, plots the first five correctly classified digits, and a manually chosen collection of misclassified digits.

For each classifier implementation, all the data was first normalized by diving all the data by 255. This brings the data into the range $\{0, 1\}$ as the pixel values of the MNIST data were in the range $\{0, 255\}$.

5.3.1 NN-classifier

Initially the NN-classifier was implemented by finding the distance between the entire test set and the entire training set using

```
scipy.spatial.distance.cdist()
```

Then the class of each element of the test set was set to correspond with the class of the closest element of the training set.

This method proved to take too long time. In fact, we were never able to finish a classification within a reasonable time frame. To speed up the calculations, both the test and training sets were divided into chunks to reduce the size of the matrices compared by the `cdist()`. Instead of comparing the entire test and training sets, smaller chunks were compared at a time. We implemented with a chunk size $M = 1000$ Doing this made the calculation time manageable, although it still took multiple minutes to perform. For each chunk of the training set, the element of the training set chunk closest to each element of the test set was found. If this distance was shorter than the previously found shortest distance it was replaced as the new

closest element. As before, the class of each element of the test set was chosen to correspond with the class of the closest element found from the training set.

5.3.2 Kmeans NN-classifier

After the training data was normalized, the template set was created by using

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=M, random_state=67)
kmeans.fit_predict(train_features)
template = kmeans.cluster_centers_.astype(np.float32)
```

This sets each of the references to the centre of one of the clusters found using the `KMeans` class from the sklearn library. Setting the `random_state` variable to an arbitrary, but consistent, integer ensures reproducibility across separate runs of the program.

The class of the template is, as mentioned in 3.3.2, chosen as the most common label within each cluster. If two labels are equally common, one is arbitrarily chosen by the numpy `argmax()` function.

After creating the template set, the classifier was implemented in almost the same way as the NN-classifier. The only difference is that, due to the already small size of the template set, it was not divided into chunks. The test-set was divided into chunks before being compared to the entire template set at once. The label of each element in the test set was chosen as the label of the closest template.

5.3.3 KNN-classifier

The distances between the training and test sets were calculated identically between the NN- and KNN-classifiers. The difference starts in that the KNN-classifier saves the shortest K distances in a numpy array. At the end, it assigns to each element of the test set the most common label amongst the K closest elements of the training set to it. Similarly to when we assigned the label to the Kmeans cluster, if two labels are equally common, one is arbitrarily chosen by the numpy `argmax()` function. For our implementation we used $k = 7$ as per the task specification.

5.4 MNIST Results

5.4.1 NN-classifier

The NN-classifier had an overall error rate of 0.0309 and took 228.9 seconds to complete the classification. The error rate is quite low, but the classifier appears to especially classify fours

as nines, threes as fives, and twos as sevens per the confusion matrix 6. 8 showcases some examples of incorrectly classifies examples. The first two are quite easy to tell which digits are represented, and are likely classified incorrectly by the NN-classifier as they look similar to the predicted if the top of the four and bottom of the nine had been connected. The next three may be difficult to read at all and the reason behind the misclassification is harder to explicitly determine, though given the digits are hard to read, one can easily imagine there are other hard to read digits in the training set that may be similar.

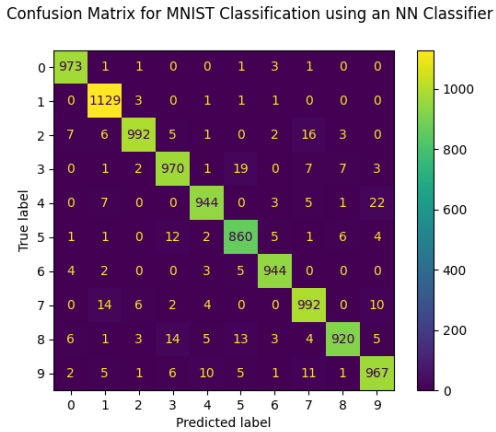
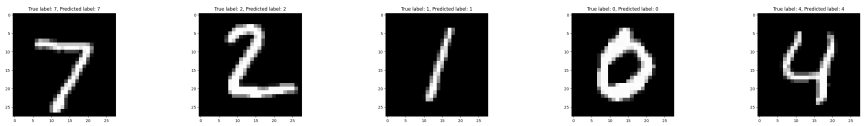
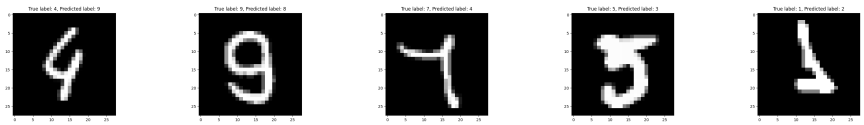


Figure 6: NN-classifier confusion matrix



(a) True: 7 **(b)** True: 2 **(c)** True: 1 **(d)** True: 0 **(e)** True: 4

Figure 7: Some correctly classified digits



(a) True: 4, Predicted: 9 **(b)** True: 9, Predicted: 8 **(c)** True: 7, Predicted: 4 **(d)** True: 5, Predicted: 3 **(e)** True: 1, Predicted: 2

Figure 8: Some incorrectly classified digits

5.4.2 Kmeans NN-classifier

The Kmeans based NN-classifier had an overall error rate of 0.1516 and took 0.2 seconds to complete the classification. This is a higher error rate than the NN-classifier, though a significantly lower classification time. Based on the confusion matrix 9 it appears that the

classifier especially confuses fours and nines, and struggles to classify threes, often mistaking them for threes or eights. Taking a look at the collection incorrectly classified digits in 11, most of them are very easy to tell what digit is actually written. This is likely a by-product of the Kmeans clustering somewhat similar numbers such as fours and nines, fives and threes, and threes and eights together.

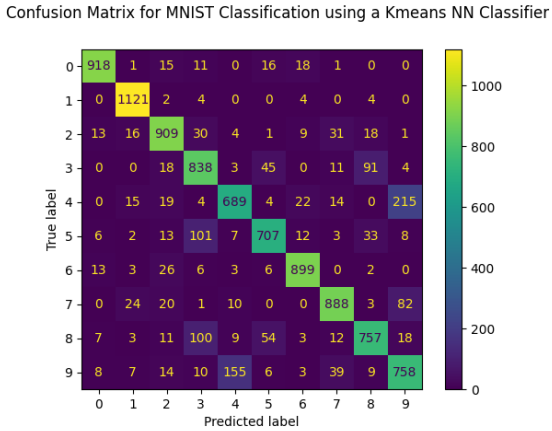
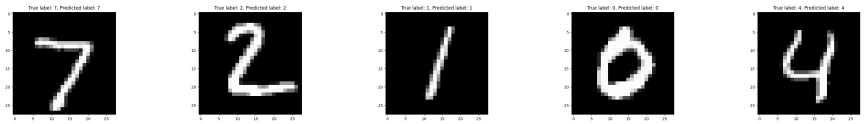
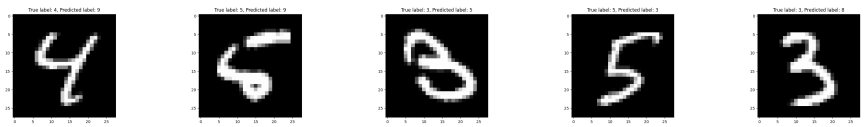


Figure 9: Kmeans NN-classifier confusion matrix



(a) True: 7 **(b)** True: 2 **(c)** True: 1 **(d)** True: 0 **(e)** True: 4

Figure 10: Some correctly classified digits



(a) True: 4, Predicted: 9 **(b)** True: 5, Predicted: 9 **(c)** True: 3, Predicted: 5 **(d)** True: 5, Predicted: 3 **(e)** True: 3, Predicted: 8

Figure 11: Some incorrectly classified digits

5.4.3 KNN-classifier

The KNN-classifer had an overall error rate of 0.0306 and took 276.5 seconds to complete the classification. This is both the lowest error rate and the longest completion time out of any of the classifiers. Overall it outperforms the NN-classifier, but notably it misclassified a higher number of ones as sevens as can be seen in the confusion matrix 12. As can be seen in the collection of incorrectly classified digits 14, the KNN-classifier still misclassifies the same four and nine as the NN-classifier. In addition, a few of the sevens that have been classified as ones can be seen.

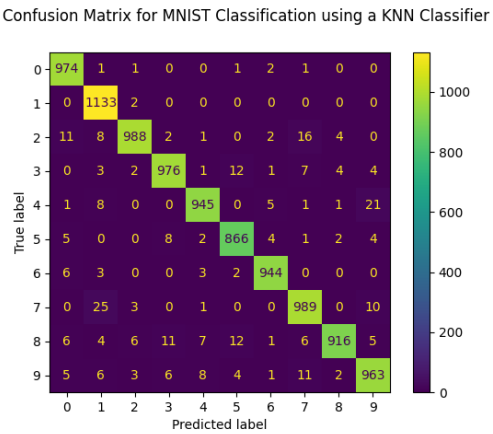
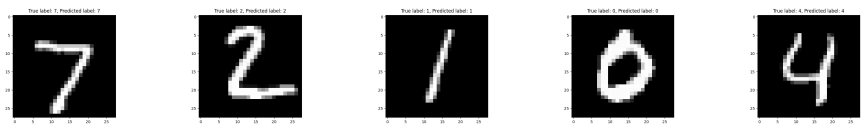
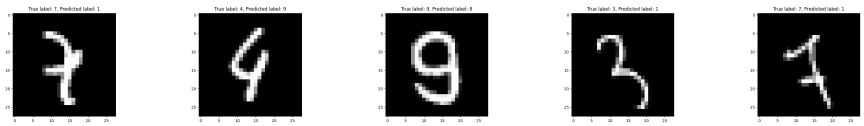


Figure 12: KNN-classifier confusion matrix



(a) True: 7 **(b)** True: 2 **(c)** True: 1 **(d)** True: 0 **(e)** True: 4

Figure 13: Some correctly classified digits



(a) True: 7, Predicted: 1 **(b)** True: 4, Predicted: 9 **(c)** True: 9, Predicted: 8 **(d)** True: 3, Predicted: 1 **(e)** True: 7, Predicted: 1

Figure 14: Some incorrectly classified digits

5.4.4 Comparison of the classifiers

Although the KNN-classifier slightly outperforms and takes slightly longer time to run than the NN-classifier, The KNN- and NN-classifiers perform similarly and take comparable times to run. The Kmeans based classifier both performs significantly worse, and runs significantly faster. An interesting thing to note are the correctly classified digits shown in fig 7, fig 10, and fig 13. As mentioned in section 5.3, only the first five correctly classified digits have been plotted. These are the exact same digits for each classifier. This implies that a collection of the same digits are easy to classify for all three classifiers.

6 Conclusion

We implemented and explored the theory behind a linear classifier. We used the linear classifier on the Iris dataset split in two different ways. For normal runs we achieved error rates of 0.0333 and 0 with most confusion being between the Iris-Versicolor and Iris-Virginica. We then explored removing features with overlap in order of most to least overlap. This resulted in deterioration of our error rates as the classifier was less able to distinguish between classes.

We implemented three variants of a template based classifier for the MNIST dataset; namely we implemented an NN-classifier, an NN-classifier with a Kmeans based template, and a KNN-classifier. The KNN-classifier slightly outperformed the NN-classifier with an error rate of 0.0306 compared to one of 0.0309 for the NN-classifier, which in turn significantly outperformed the Kmeans based classifier which achieved an error rate of 0.1516. The Kmeans based classifier on the other hand ran significantly faster than either the NN- or Knn-classifier.

References

[1] Magne H. Johnsen, *Classification* 18. Dec. 2017

[2] R. van den Boomgaard, "k-Means Clustering," *Machine Learning Lecture Notes*, 2021. `https://staff.fnwi.uva.nl/r.vandenboomgaard/MachineLearning/LectureNotes/Clustering/kmeans.html`